Simulation d'une usine en V : premiers enseignements

Application de la Théorie des Contraintes (TOC) avec SimPy

Projet IndEng

9 novembre 2026

Résumé

Nous simulons une usine de type **V** (un flux amont partagé diverge vers deux lignes produit) avec le moteur à événements discrets `indeng` (SimPy). Vingt exécutions Monte-Carlo d'un scénario à demande variable permettent d'identifier le goulot d'étranglement, de mesurer son impact sur les délais et les retards, et d'illustrer les principes de la Théorie des Contraintes : le débit de l'usine est dicté par sa ressource contrainte (CCR), et non par la somme des capacités locales.

1 Le modèle simulé

L'usine en V comporte trois postes (figure 1) :

— `prep` (capacité 2) : poste amont partagé par les deux produits ;
— `line_a` (capacité 1, temps 3) : produit **P1** ;
— `line_b` (capacité 1, temps 6) : produit **P2**.

La demande est un processus de Poisson (taux 0,02 commande/unité de temps) portant sur P1 ou P2, avec des quantités aléatoires entre 5 et 20. Les temps de changement de série sont de 1 unité par défaut et de 3 unités lors d'un changement de produit sur `prep`. L'ordonnancement est FIFO, l'horizon de simulation est 2000 unités de temps, et chaque scénario est rejoué 20 fois avec des graines différentes.

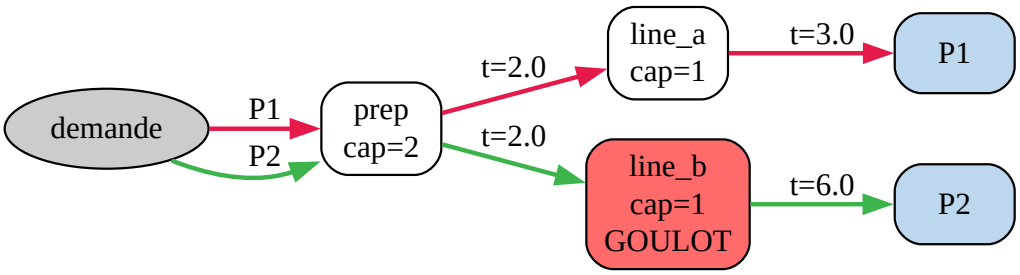


FIGURE 1 – Topologie de l'usine en V (Graphviz). Le goulot `line_b` est surligné en rouge. Les arêtes sont colorées par produit et étiquetées par le temps d'opération.

2 Résultats

Le tableau 1 regroupe les indicateurs clés (moyenne ± écart-type sur 20 runs). Les figures 2 à 4 illustrent une exécution représentative.

TABLE 1 – KPIs du scénario V (20 runs Monte-Carlo, horizon 2000).

Indicateur	Moyenne	Écart-type
Utilisation `prep`	0,267	0,040
Utilisation `line_a`	0,375	0,094
Utilisation `line_b` (goulot)	0,668	0,147
Throughput P1 (unités)	249,9	62,8
Throughput P2 (unités)	222,8	48,9
Throughput total	472,6	75,8
Lead time P1	80,5	10,5
Lead time P2	166,1	44,3
Makespan	1959,8	31,6
Commandes terminées	38,3	5,9
WIP moyen	2,39	0,77
Commandes en retard	8,5	5,3
Taux de retard	21,4 %	10,9 %

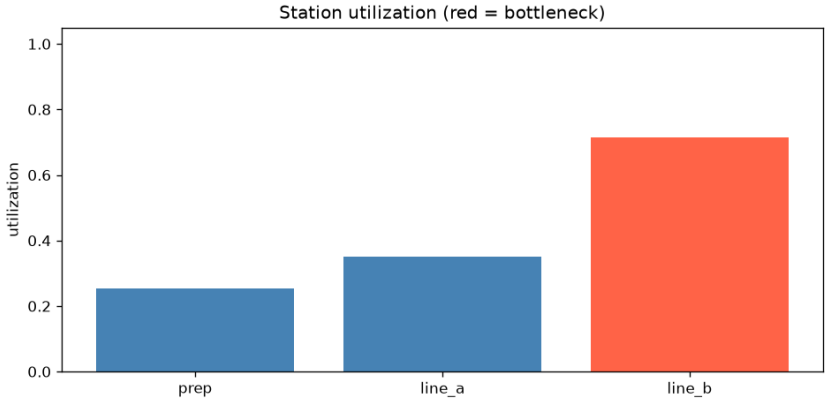


FIGURE 2 – Utilisation des postes : `line_b` (rouge) est la ressource contrainte, nettement devant les autres.

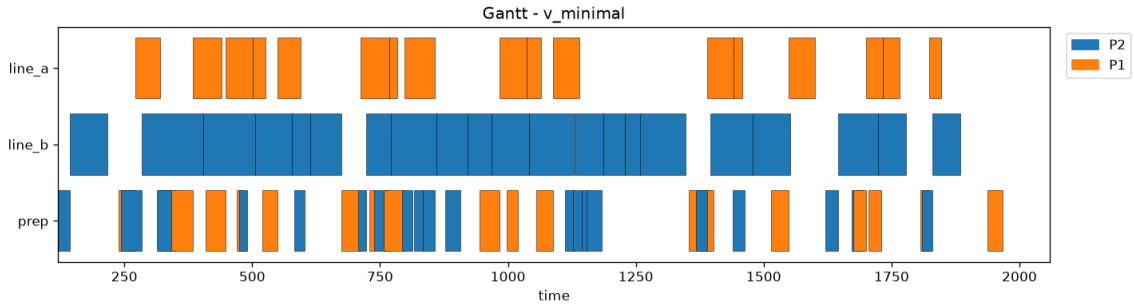


FIGURE 3 – Diagramme de Gantt d'un run : `prep` alterne les produits (setups), les lignes sont presque continûment occupées par leur produit.

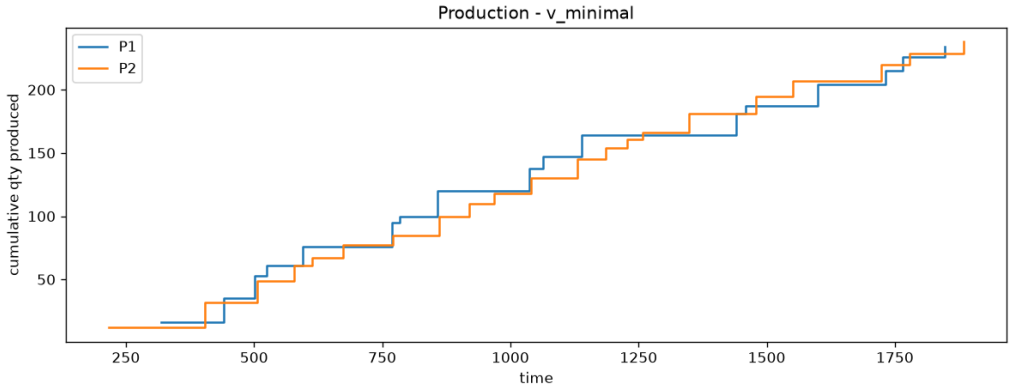


FIGURE 4 – Production cumulée de P1 et P2 : des pentes comparables, le débit étant cadencé par le goulot.

3 Que nous apprend cette simulation ?

3.1 Un seul poste dicte le débit de toute l'usine

Le goulot `line_b` est détecté comme poste le plus utilisé dans 19 runs sur 20 (utilisation 0,67 contre 0,27–0,38 pour les autres). C'est la *ressource contrainte* (CCR) au sens de la TOC : **le throughput de l'usine est celui du goulot**, pas la somme des capacités des postes. Accélérer `prep` ou `line_a` n'augmenterait pas la production globale.

3.2 Les files se forment devant le goulot, pas ailleurs

Le lead time de P2 (166) est environ le double de celui de P1 (80) : les commandes de P2 s'accumulent en amont de `line_b`. Le diagramme de Gantt montre d'ailleurs des lignes quasi pleines, signe d'un système tiré par la contrainte. Le WIP moyen reste faible (2,4) : il n'y a pas d'en-cours inutile qui s'accumule devant les postes non contraints.

3.3 Sans buffer, la contrainte génère des retards

Avec une échéance fixée à 2 fois le temps de passage nominal et aucun buffer de temps devant le goulot, 21 % des commandes sont livrées en retard. C'est le rôle du **Buffer** dans la méthode DBR (Drum-Buffer-Rope) : protéger le goulot par un buffer de temps pour absorber la variabilité de la demande et des postes amont.

3.4 La variabilité impose le Monte-Carlo

La demande étant aléatoire, les écarts-types sont élevés (throughput total : $\pm$ 76 unités, taux de retard : $\pm$ 11 points). Une seule exécution ne serait pas représentative : les 20 runs avec graines fixées donnent des résultats reproductibles et des intervalles de confiance exploitables.

3.5 Le levier d'amélioration est la contrainte elle-même

Pour augmenter le débit de l'usine, la TOC prescrit d'*élever la contrainte* : réduire le temps de `line_b` ($6 \rightarrow 3$, ou capacité 2) aura un effet direct sur le throughput total et les retards, alors que des gains sur `prep` n'en auraient quasiment pas (et créeraient du WIP inutile).

4 Limites et perspectives

Ce premier modèle reste volontairement simple : pas de pannes ni de maintenance, des setups symétriques. L'archétype **A** est implémenté : des composants sont produits sur des lignes dédiées puis assemblés (nomenclature BOM) en deux produits finis P1 et P2 (figure 5) ; dans ce scénario, le poste partagé `decoupe_b` (présent dans les deux nomenclatures) devient la contrainte. Les prochaines étapes sont l'archétype **T** (scaffoldé, figure 6), la règle d'ordonnancement **DBR** (scaffoldée dans `indeng/scheduling`), et des buffers de temps configurables devant le goulot pour comparer les politiques de protection.

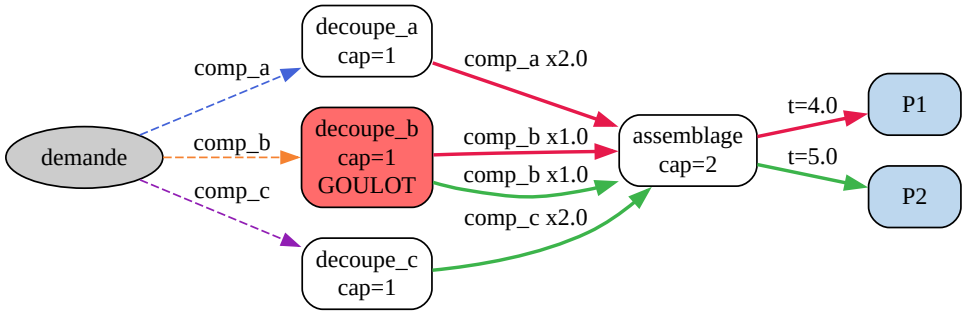


FIGURE 5 – Archétype A : les composants (arêtes en pointillés, quantités sur les arêtes de nomenclature) convergent vers `assemblage`, qui produit deux produits finis P1 et P2. Le goulot `decoupe_b` est surligné en rouge.

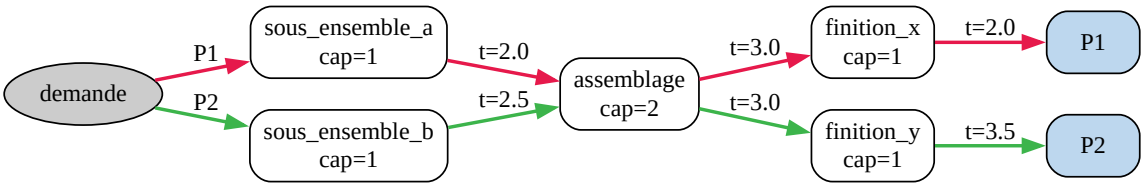


FIGURE 6 – Archétype T (démonstration) : convergence vers l'assemblage puis divergence vers les finitions.

5 Reproductibilité

```
pip install -e .
python -m indeng run config/scenarios/v_minimal.yaml --runs 20 --out out
python -m indeng graph config/scenarios/v_minimal.yaml --out out --highlight line_b
pytest   # 23 tests
```

Les sorties versionnées (KPIs, événements, graphiques, topologies) sont dans `results/`.